

## Modernizing LARS:

### Open-source, modular, and programmatically flexible scenario replay

Raphael Huang<sup>1</sup> and Nick Goberville<sup>2</sup>

<sup>1</sup>*Science Undergraduate Laboratory Internships (SULI), Argonne National Laboratory*

<sup>2</sup>*Argonne National Laboratory*

(Dated: 11 December 2025)

The Longitudinal Automated Replay System (LARS) was modernized to support the energy consumption testing of Connected and Automated Vehicles (CAVs), transforming a closed-source system into an open-source, modular, and flexible architecture. Originally built on proprietary hardware and software such as a Speedgoat real-time target machine and Simulink/MATLAB framework, LARS was well-suited for its initial application as a lead vehicle in the EcoCAR EV Challenge. However, its inflexible architecture inhibited commercialization potential and extensibility for broader research.

To address these limitations, the system was rebuilt from the ground up on a new open-source framework using Python and the Robot Operating System (ROS). This architectural shift and the creation of a modular system schema with precisely-defined custom message types enhanced the system’s robustness and scalability. Additionally, key components were developed to improve functionality, including an intuitive smart dashboard for real-time data visualization and control, a state machine with comprehensive safety features, and a flexible vehicle interface for the Cadillac LYRIQ designed to work with any DBC file. The new system’s flexibility and powerful tools enable a variety of new use cases, allowing vehicle researchers to rapidly design and test new energy management and controls strategies.

## I. INTRODUCTION

Electric vehicles (EVs) are quickly becoming the most dominant vehicle type in the global transportation market. In 2024, EVs accounted for more than 20% of all cars sold worldwide.<sup>1</sup> By 2035, 1-in-4 cars will be an EV.<sup>1</sup> This ballooning demand for all categories of EVs also means there is a growing appetite for the features that Connected and Automated Vehicles (CAVs) bring.

However, closing the development loop for CAV features has long been difficult. Corporations in the CAV space keep tight control over their software-hardware integration techniques, and the open-source CAV community, which includes most researchers at universities and national labs, has largely been left in the dust.

LARS was initially built to partially address these issues. Primarily motivated by the collegiate Argonne-led EcoCAR EV Challenge which needed a lead car with repeatable drivetrace following to analyze the performance of the student-designed trailing car, the initial version of LARS was developed in a tight timeframe. Therefore, it used an architecture that leveraged existing proprietary and closed-source hardware and software technologies developed by sponsors of the EcoCAR competition. The hardware of choice was the Speedgoat real-time target machine, and the software of choice was Mathworks’ Simulink and MATLAB. This software-hardware tandem was tightly integrated due to a collaboration between Speedgoat and Mathworks that provided a link between the Speedgoat’s CAN card input and Simulink’s real-time kernel. Although this tandem worked well for its initial use case, the original intention was for LARS to be vehicle-agnostic, hardware-agnostic, and easy for the wider CAV research community to access and use; all things not possible when the entire system is confined to proprietary and closed-source hardware and software.

Rebuilding LARS in ROS, the Robot Operating System ar-

chitecture, is our response to the shortcomings of the previous version of LARS. By starting from scratch while borrowing structures and design motifs that worked well from the previous version of LARS, we were able to engineer a new scenario replay system that is modular, programmatically flexible, and open-source, moving us towards our goal of making LARS as extensible as possible.

## II. SYSTEM DESIGN & ARCHITECTURE

With the guiding principles of using open-source tools to create a modular and programmatically flexible architecture in mind, we analyzed the previous implementation and its shortcomings to arrive at a new set of hardware and software technologies.

### A. New Technology Choices

On the hardware side, the previous implementation used the Speedgoat real-time target machine. We wanted to create an architecture that would run on a wider variety of computers, so we made the choice to transition to ROS, which means our new system runs on any Linux machine with a CAN card.

To interface with the hardware, the shift to ROS and Linux gives the developer flexibility to choose the soft real-time CAN interface most comfortable for them. We chose to use Kvaser’s `canlib` for interfacing with the vehicle CAN, but theoretically any other open-source CAN interface (e.g. `socketcan`) would work just as well.

Finally, on the software side, we chose to use ROS’s support for Python through the `ament_python` build type, which enables the use of the myriad open-source Python libraries for development. We chose to use Dash/Plotly for a low-code frontend and various open-source libraries from inside and

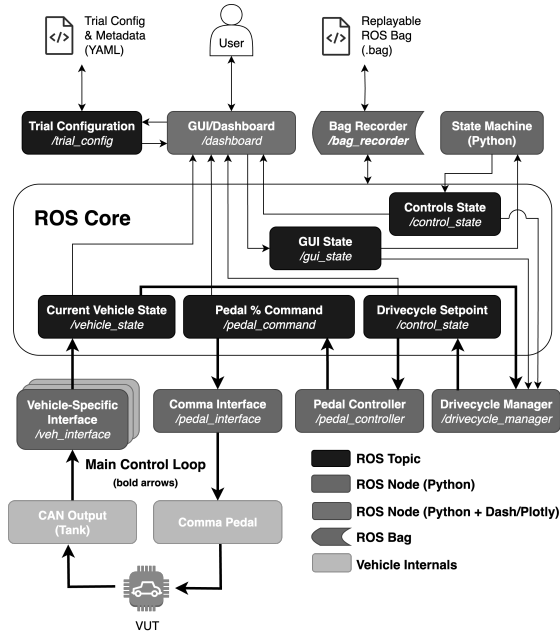


FIG. 1. LARS in ROS schema.

outside the ROS ecosystem such as `rosbag`, `transitions`, and `pandas` for the backend.

## B. Architecture Overview

A diagram of the overall architecture of LARS in ROS is shown in Fig. 1. Shown are the core ROS nodes and topics required for a typical scenario replay trial. ROS *nodes* send *messages* to each other through *topics*, which nodes either have a *publisher* (write) or *subscriber* (read) relationship with. These relationships are shown in the figure by the directional arrows.

The schema and its nodes can be split up into four core functional groups that mirrors the previous implementation of LARS. The *main control loop*, which includes the **Vehicle-Specific Interface**, **Comma Interface** and **Pedal Controller** nodes, make up the closed-loop system responsible for drivetrace following. The *dashboard* consists of the **GUI/Dashboard** node which allows the human user to interact and introspect the system, allowing for monitoring of vehicle status and speed as well as overrides to make sure the human is always in control. The *state machine* (**State Machine** node) monitors the state of the vehicle and GUI to coordinate human interaction with the controls and to make smart safety decisions. Finally, the *logging* system, including the **Trial Configuration** topic and **Bag Recorder** node, allow trials to be documented and replayed at a later time for system and data analysis.

One primary advantage of this architecture is the abstraction of *custom message types* that is built into ROS. By requiring all nodes that publish and/or subscribe to a topic to use a strongly-typed data structure (similar to a C/C++ `struct`), we achieve "plug-and-play" functionality where each node's implementation is completely isolated from every other node.

As long as they agree on how they want to communicate, each node can run independently to one another.

## III. IMPLEMENTATION

This section details the implementation of each core part of this new version of LARS, both software and hardware.

### A. Hardware

The development of LARS in ROS was done on the Tank 870 from IEI Integration Corp. running Ubuntu Desktop 24.04 with a Kvaser CAN card installed. LARS in ROS should run on any system that supports ROS, which includes native binaries for Ubuntu, Windows 10, and RHEL 9, as well as macOS if ROS is built from source.

The vehicle used to test LARS in ROS was the 2024 Cadillac LYRIQ. The CAN network of the LYRIQ was snooped by the Tank, which allowed the software to interpret and read the vehicle CAN signals given a DBC given by the OEM.

To control the acceleration of the vehicle, the open-source comma pedal was used, developed by comma.ai. The comma pedal is a gas pedal interceptor, and includes open-source PCB blueprints and code.

### B. Software

The software for LARS is implemented as a collection of modular nodes within the Robot Operating System (ROS 2 Jazzy) framework, developed primarily in Python. As outlined in the architecture in Sec. II B, these nodes operate independently and communicate using ROS's strongly-typed publisher/subscriber messaging system. This modularity ensures that components can be developed, tested, and updated in isolation. The following sections detail the implementation of the five core software modules: the **Dashboard**, the **State Machine**, the **Vehicle-Specific Interface**, the **Pedal Controller**, and the **Logging** system.

#### 1. Dashboard

The **dashboard** is the frontend of the LARS in ROS system and allows the developer/operator to both introspect the system and vehicle states, as well as starting and stopping trials, and altering current trial configurations. See Fig. 4.

Two Python libraries, Dash and Plotly, were used to generate the frontend. Dash generates a React app that runs using Flask, and creates low-effort frontends for data analysis applications.

The dashboard subscribes to nearly all topics in the LARS in ROS architecture – including `/trial_config`, `/vehicle_state`, `/control_state`, `/drivecycle_setpoint`, and `pedal_command` – in order

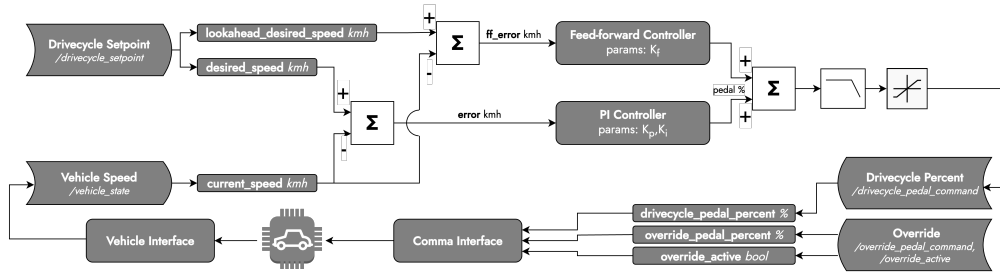


FIG. 2. P + I + F controller implemented in LARS in ROS.

to gather all system data in a single place and display them in a visually appealing way.

The node only publishes to two topics: `/trial_config` and `/gui_state`. `/trial_config` is logged for posterity and reuse in future trials, and `/gui_state` is subscribed by several other nodes to allow the user to control the system state and trigger an emergency stop if necessary.

## 2. State Machine

The primary purpose of the **state machine** is to ensure safe operation of LARS. It utilizes the `pytransitions` library to streamline state machine creation and ensure no race conditions put the state machine in an invalid state.

The state machine node subscribes to `/vehicle_state` and `gui_state` to transition states when certain conditions are fulfilled. See Fig. 5 to see how those conditions are displayed to the user. Signal arbitration always prioritizes `/vehicle_state`; for example, when the brake is pressed during a trial where LARS is in full control of the vehicle, the control state will immediately deactivate all overrides to cede control over to the operator.

`/control_state` is the only topic published to by the state machine.

## 3. Vehicle-Specific Interface

In our current implementation of the **vehicle-specific interface**, we utilize `canlib`, which is an open-source Python wrapper for Kvaser CANlib. We use `canlib` to sift through all the signals flowing on the CAN bus and translate signals from the vehicle CAN network given a database CAN file for the specific vehicle. Since this process is computationally intense and needs to operate at soft real-time rates, the Python threading module was used to run the `canlib` integration on a separate thread.

After translation, the decoded fields such as the vehicle speed, acceleration, and gear are published to the `/vehicle_state` topic.

## 4. Pedal Controller and Drivecycle Manager

The current controller that will ship with this initial version of LARS in ROS is a proportional, integral, feed-forward controller which is typical for this type of controls application. A diagram of the controller is shown in Fig. 2. The **pedal controller** implements this controller alongside a low-pass filter, saturation gate, and other reset logic to prevent integral windup and smooth out the controller's output. The gains for each term in the controller,  $K_P$ ,  $K_I$ , and  $K_F$  for the proportional, integral and feed-forward terms respectively, are implemented as ROS parameters and can be adjusted while the controller is running.

The pedal controller node subscribes to the `/drivecycle_setpoint` topic which is published by the **drivecycle manager**, a simple node that reads from a user-provided CSV drivetrace and interpolates between points to continuously publish speed setpoints. The drivecycle manager also responds to the `/gui_state` to ensure the system responds to e-stops and the end of each trial.

The node then transmits its resulting output to the `/pedal_command` topic, to be read by the pedal interface.

## 5. Logging

The **bag recorder** node and `/trial_configuration` topic are the main mechanisms through which LARS in ROS logs data for post-trial analysis and debugging. Although these two logging mechanisms do not necessarily provide comprehensive post-trial analysis of how well the controller performed or errors that occurred, the bag recorder utilizes ROS's `ros2 bag` feature which allows for precise replay of all topics in a compressed format. Then, at later time, the developer can create any number of analysis nodes to read in topics and analyze their contents as if the trial were running live.

After every trial, the trial config YAML and the replayable ROS `.bag` file are bundled and stored together in the users' file system.

## IV. SYSTEM VALIDATION & RESULTS

To validate the performance and demonstrate the success of LARS in ROS, a multi-stage testing process was conducted. The system was first validated in two progressive simulation environments to safely test and tune the closed-loop controller before deployment. Following these successful simulations, the system was integrated into the Cadillac LYRIQ test vehicle, where it demonstrated it was capable of robust and accurate drivetrace following in real-world conditions. This section presents the results from both the simulations and on-road testing, and also discusses the emergent benefits and new capabilities enabled by the flexible, open-source architecture.

### A. Simulations

The two simulations implemented to close the loop early were an ideal controller that had no simulated resistances, and a quadratic drag controller that simulates quadratic air resistance and linear rolling resistance. Although these controllers were far from realistic, they helped ensure the controllers had relatively sane behaviors before deployment on the vehicle.

### B. Vehicle Testing

During vehicle testing, a **pedal override** GUI was first used to ensure the comma pedal interface was working properly. The pedal override was a separate node and Dash app from the main dashboard, built solely for debugging system oddities and to ensure the system could accurately send pedal commands to the vehicle.

A gradual approach was used to test various safety mechanisms such as the state machine and human overrides were working before LARS was allowed to take full control of the vehicle.

During live vehicle testing, after tuning the controller's parameters, we saw fairly accurate drivetrace following over a simple 40 second "trapezoidal" drivetrace, with the aggregate root mean square error averaging 152.29.

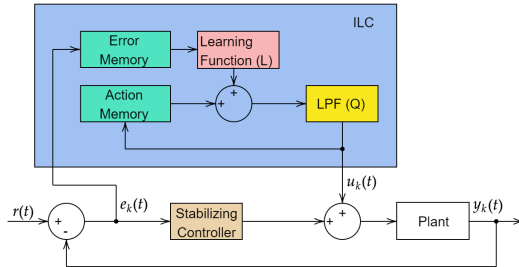


FIG. 3. One implementation of an iterative learning controller.<sup>2</sup>

## C. New Capabilities

Some emergent benefits came about due to the new programmatically flexible architecture. They are detailed below.

### 1. Iterative Learning Control

Iterative learning control (ILC) is a control strategy that improves the performance of systems executing the same task repeatedly. Much like a person learning a motor skill through practice, ILC refines its control action by learning from the tracking error of the previous attempt. See Fig. 3 for a block diagram of ILC.

Specifically, for a given trajectory over a fixed time duration, the error signal from the entire previous trial (iteration  $k$ ) is used to calculate a correction to the control input for the current trial (iteration  $k+1$ )<sup>3</sup>. This relationship is represented by the equations below, where  $u_k$  is the  $k$ th trial's control output,  $\Gamma$  is the learning function's gain, and  $e_k(t)$  is the error from the  $k$ th trial (desired - actual):

$$u_{k+1}(t) = \underbrace{u_{PI,k+1}(t)}_{\text{Real-time}} + \underbrace{u_{FF}(t)}_{\text{Static FF}} + \underbrace{u_{ilc}(t)}_{\text{Learning FF}} \quad (1)$$

$$\text{where, } u_{ilc}(t) = \Gamma \cdot e_k(t) \quad (2)$$

A hybrid control strategy was implemented, combining a real-time PI feedback controller with an iteratively learned feed-forward component to progressively eliminate repeatable drivetrace following errors.

The modernized LARS architecture is uniquely suited for implementing ILC. The system's ability to precisely log, reset, and replay drive cycles provides the necessary iterative framework. By analyzing the speed error from a completed trial, an offline ILC Python script generated a set of biases that modified the pedal command biases for the next run, enabling the vehicle to track the desired speed trace with higher accuracy. In our trials, after only one iteration of ILC, the aggregate root mean square error decreased to 53.158, compared to 152.29 mentioned in Sec. IV B.

### 2. Remote Access

The transition to a Linux-based architecture inherently unlocked powerful remote access capabilities unavailable on the previous proprietary system.

Operators can now securely connect to the in-vehicle computer's command line via SSH for system management and debugging. Furthermore, since the dashboard is a web application, it can be exposed on a local network port. This allows a copilot or engineer to monitor and control a test in real-time from a separate laptop connected via Ethernet. While not yet implemented, the platform also readily supports a VNC server, which would enable full remote graphical desktop access for more complex off-site operations.

## V. CONCLUSIONS & FUTURE WORK

This work successfully modernized the Longitudinal Automated Replay System (LARS), transforming it from a rigid, proprietary platform into a flexible and open-source tool for advanced Connected and Automated Vehicle research. By re-architecting the system using the Robot Operating System (ROS), Python, and standard Linux-based hardware, we created a modular framework that is both vehicle- and hardware-agnostic. The new system was validated through simulation and on-road testing, demonstrating accurate drivetrace following on a 2024 Cadillac LYRIQ.

Future development will focus on expanding the system's capabilities and improving its accessibility. Key priorities include creating automated post-trial analysis tools to quantify controller error and vehicle energy efficiency, expanding the library of officially supported controllers and vehicle interfaces to increase out-of-the-box utility, and developing comprehensive documentation, user guides, and tutorials to make it easier for new developers to start adding their own modules to LARS.

Ultimately, these efforts aim to lower the barrier to entry for repeatable, real-world CAV testing. By providing a powerful and accessible open-source platform, we hope to foster a community of users and contributors across universities, national laboratories, and industry, accelerating the development and validation of next-generation vehicle control and energy management strategies.

## ACKNOWLEDGMENTS

This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Science Undergraduate Laboratory Internships (SULI) program.

## WORKS CITED

- <sup>1</sup>I. E. Agency, "Global ev outlook 2025: Expanding sales in diverse markets," Electric Vehicles Initiative (2025).
- <sup>2</sup>ILC, <https://www.mathworks.com/help/slcontrol/ug/model-based-ilc-for-mimo-model.html>.
- <sup>3</sup>M. N. Svante Gunnarsson, "A short introduction to iterative learning control," (1997).

## Appendix: Diagrams and Screenshots

### LARS Smart Dashboard

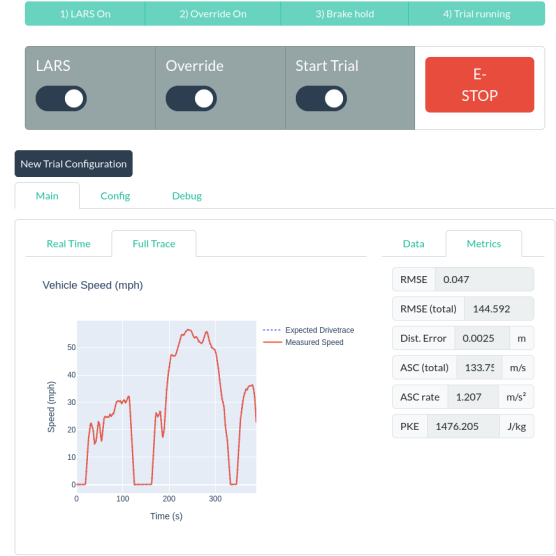


FIG. 4. The LARS dashboard GUI during a full trial. Key elements include the real-time speed plot (bottom left), system state indicators (top), and error metrics (bottom left).

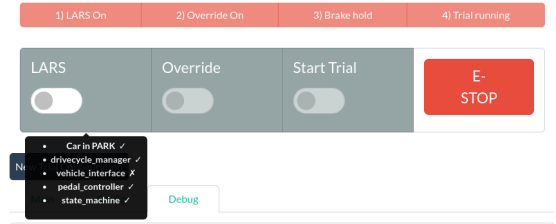


FIG. 5. The control bar on the main GUI tab. Top bar indicates current states, and switches below allow user to control system state once certain conditions shown in the dropdown are fulfilled.

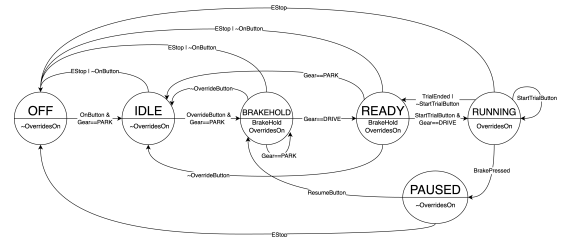


FIG. 6. A diagram representing the state machine implemented in LARS in ROS.